University of Science - VNUHCM

Faculty of Information Technology

Software Testing

HW7 - API testing

Software Testing Project Report

Authors:
Trần Thị Cát Tường
(22127444)
ttctuong22@clc.fitus.edu.vn

Supervisors:
Teacher Trần Duy Hoàng
Teacher Hồ Tuấn Thanh
Teacher Trương Phước Lộc

August 25, 2025

Table of Contents

1 Group Information

Group ID: 07

Member Name	Student ID	Assigned Features	Status
Cao Uyển Nhi	22127310	Payment	Done
		Brands	Done
		Favourite	Done
Lưu Thanh Thuý	22127410	Sign In	Done
		Add User	Done
		Edit User	Done
Nguyễn Phước Minh Trí	22127424	Create product	Done
		Edit product	Done
		Get related product	Done
Võ Lê Việt Tú	22127435	Register	Done
		Get all products	Done
		Get product	Done
Trần Thị Cát Tường	22127444	Send Contact	Done
		Create Category	Done
		Edit Category	Done

2 API Testing Summary

2.1 Scope and APIs Under Test

- **Base URL:** `http://localhost:8091`
- **Default header:** `Content-Type: application/json`
- **Auth header (when required):** `Authorization: Bearer <access_token>`
- **Login endpoints:**
  - **User:** `POST /users/login` with

```
{
  "email":"customer@practicesoftwaretesting.com",
  "password":"welcome01"
}
```

  - **Admin:** `POST /users/login` with

```
{
   "email":"admin@practicesoftwaretesting.com",
   "password":"welcome01"
}
```

2.1.1 API 1 - Send Contact Message

Endpoint: `POST /messages`

Purpose: Create a contact message as a logged-in **user** or a **guest**.

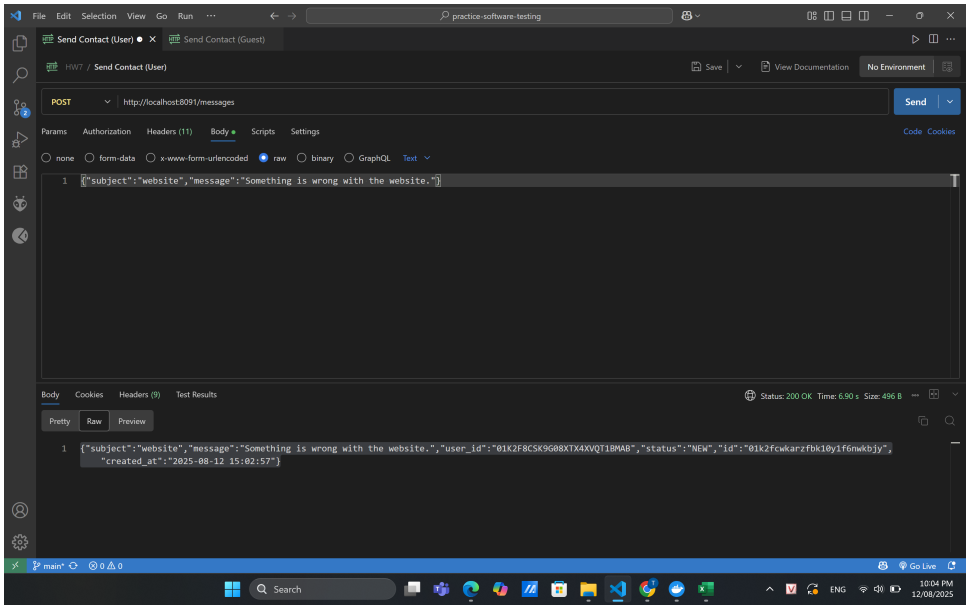

Figure 1: Send contact

Requirements and headers

- **User flow:** must log in and send `Authorization: Bearer <userToken>`.

- **Guest flow:** no Authorization header.
- Always send `Content-Type: application/json`.

Request examples

- **User**

```
{
  "subject": "website",
  "message": "Something is wrong with the website."
}
```

- **Guest**

```
{
  "name": "John Doe",
  "email": "john@doe.example",
  "subject": "website",
  "message": "Something is wrong with the website."
}
```

Key expectations and validations:

- Response includes `id`, `name`, `slug`, `parent_id`, `created_at`.
- `parent_id` must reference an existing category and must not equal the new category's own `id`.
- `name` required, max 120 chars, unique.
- `slug` required, lowercase, URL-safe, unique.

Equivalence Partitioning and BVA:

- `parent_id`: omitted/valid existing vs non-existent/equal-to-own-id.
- `name`: lengths 0, 1, 120, 121, duplicate vs unique.
- `slug`: lowercase/URL-safe/unique vs uppercase/spaces/specials/duplicate.
- Payload/format negatives: `text/plain`, malformed JSON, unknown fields.

2.1.2 API 2 - Create Category

Endpoint: `POST /categories`

Purpose: Create a new category (root or with a parent).

Requirements and headers

- Must log in as **admin** include `Authorization: Bearer <adminToken>`.
- `Content-Type: application/json`.

Request example

```
{
  "parent_id": "01K2F8CT583ZAYT9MVADFN440D",
  "name": "Audio",
  "slug": "audio"
}
```

Key expectations and validations:

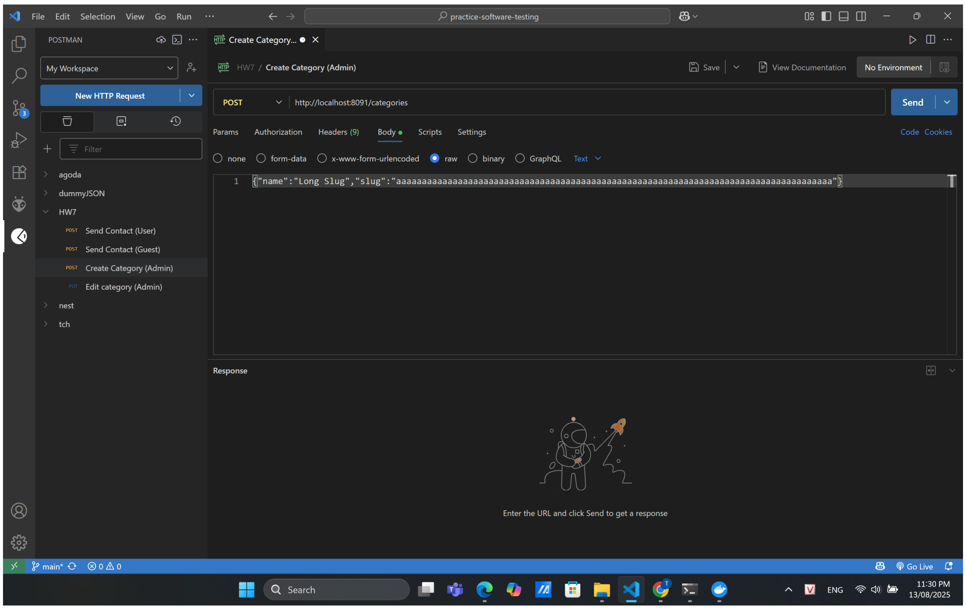

Figure 2: Create category

- `{categoryId}` must exist, response reflects updated fields, `parent_id` can be set or cleared.
- `name` required, $\leq$ 120 chars, not duplicating another category's name.
- `slug` required, lowercase, URL-safe, unique, keeping the current slug is allowed.
- `parent_id` must exist, cannot equal `{categoryId}`, should not create cycles.

Equivalence Partitioning and BVA:

- `name`: lengths 0, 1, 120, 121, duplicate-of-other vs unique, unicode name.
- `slug`: valid vs uppercase/spaces/underscore/specials/duplicate-of-other, very long slug, trimming spaces, unchanged slug.
- `parent_id`: valid existing vs non-existent/equal-to-self/cycle.

2.1.3 API 3 - Update Category

Endpoint: `PUT /categories/{categoryId}`

Purpose: Update `name`, `slug`, and/or `parent_id` of an existing category.

Requirements and headers

- Must log in as **admin** include `Authorization: Bearer <adminToken>`.
- `Content-Type: application/json`.
- `{categoryId}` must exist (otherwise 404).

Request example

```
{
  "parent_id": "01K2HXEXGVBEQQZD2B3GCVWSPJ",
  "name": "new categoryyyy",
  "slug": "new-categoryyy"
}
```

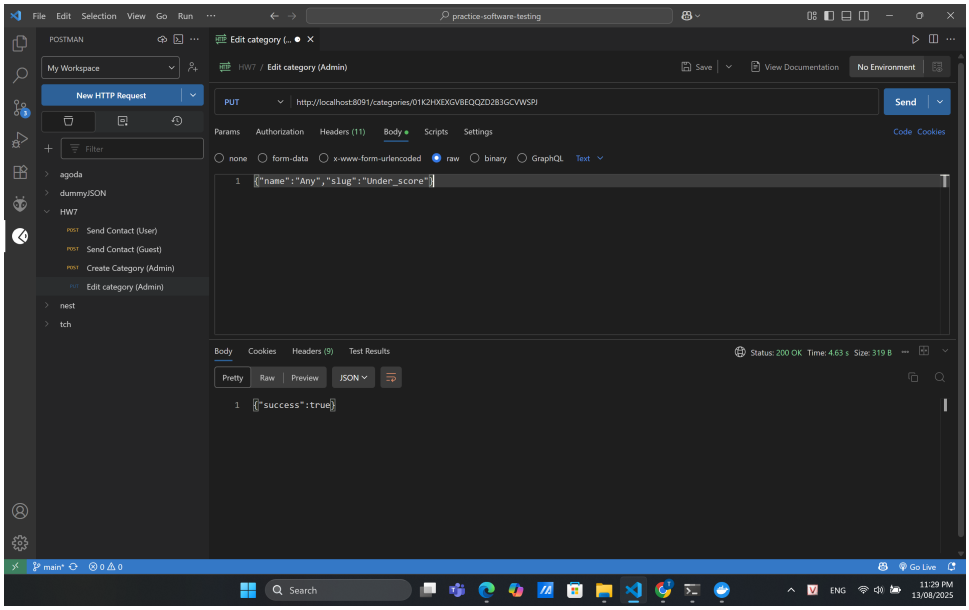

Figure 3: Update Category

Key expectations and validations

- `{categoryId}` must exist, response reflects updated fields (`name`, `slug`, `parent_id`).
- Same input rules as Create:
  - `name` required, $\leq$ 120 chars, unique among categories.
  - `slug` required, lowercase, URL-safe, unique.
  - `parent_id` optional, if provided, must reference an existing category.
- **Parent rules**: cannot set `parent_id` to itself, cannot set parent to its own descendant.
- **Slug uniqueness**: keeping the current slug is allowed, duplicating another category's slug is not.
- **Parent clearing**: sending `"parent_id": null` makes the category a root category.
- **ID consistency**: if body contains `id`, it must match `{categoryId}`.

2.2 AI-Assisted Test Case Generation

To accelerate test design and improve coverage, I applied Generative AI (ChatGPT) to create an initial draft of our test cases based on pre-defined validation rules and API specifications.

2.2.1 Approach

1. **Define criteria and constraints**
   - Required and optional fields.
   - Data validation rules (length limits, formats, uniqueness).

- Equivalence Partitioning, Boundary Value Analysis.

2. **Provide detailed context and prompt to the AI model**
   - API name, endpoint, HTTP method.
   - Request/response format.
   - All field constraints.
   - Example payloads.
   - Desired output format.

2.2.2 Prompt

API 1 - Send Contact Message

```
You are a software testing assistant. I will provide API details,
request/response examples, and field validation rules. Generate a set
of test cases in table format with the following columns: Test Case ID,
Title, Preconditions, Inputs, Expected Result. Include both valid
and invalid cases using Equivalence Partitioning and Boundary Value
Analysis. Cover all business rules and edge cases.

API: POST /messages
Purpose: Create a contact message as a logged-in user or a guest.
User flow: must log in and send Authorization: Bearer <userToken>.
Guest flow: no Authorization header.
Fields:
- For user: subject (required), message (required, 50-250 chars).
- For guest: name (required), email (valid format), subject (required),
message (required, 50-250 chars).
Content-Type: application/json.
Example request (user):
{
  "subject": "website",
  "message": "Something is wrong with the website."
}
Example request (guest):
{
  "name": "John Doe",
  "email": "john@doe.example",
  "subject": "website",
  "message": "Something is wrong with the website."
}
```

API 2 - Create Category

```
You are a software testing assistant. I will provide API details,
request/response examples, and field validation rules. Generate a set
of test cases in table format with the following columns: Test Case ID,
Title, Preconditions, Inputs, Expected Result. Include both valid
and invalid cases using Equivalence Partitioning and Boundary Value
Analysis. Cover all business rules and edge cases.
```

```
API: POST /categories
Purpose: Create a new category. Requires admin login.
Fields:
- parent_id: optional, must reference an existing category, cannot equal
its own ID.
- name: required, $\leq$ 120 chars, unique.
- slug: required, lowercase, URL-safe (hyphens), unique.
Example request:
{
  "parent_id": "01K2F8CT583ZAYT9MVADFN440D",
  "name": "Audio",
  "slug": "audio"
}
```

API 3 - Update Category

```
You are a software testing assistant. I will provide API details,
request/response examples, and field validation rules. Generate a set
of test cases in table format with the following columns: Test Case ID,
Title, Preconditions, Inputs, Expected Result. Include both valid
and invalid cases using Equivalence Partitioning and Boundary Value
Analysis. Cover all business rules and edge cases.

API: PUT /categories/{categoryId}
Purpose: Update name, slug, and/or parent_id of an existing category.
Requires admin login.
Fields:
- parent_id: optional, must reference an existing category, cannot equal
{categoryId}, must not create cycles; null to clear.
- name: required, $\leq$ 120 chars, unique.
- slug: required, lowercase, URL-safe (hyphens), unique.
ID consistency: if body contains id, must match {categoryId}.
Example request:
{
  "parent_id": "01K2HXEXGVBEQQZD2B3GCVWSPJ",
  "name": "new categoryyyy",
  "slug": "new-categoryyy"
}
```

3. **Review and refine**
    - Manually checked AI-generated cases to remove duplicates, align with actual system behavior, and add missing scenarios.
    - Integrated valid cases into the main test suite for execution in Postman.

2.3 Postman workflow

API tests were executed in the Postman extension within Visual Studio Code

1. **Create the collection**

- Make a collection named HW7.
- Add request

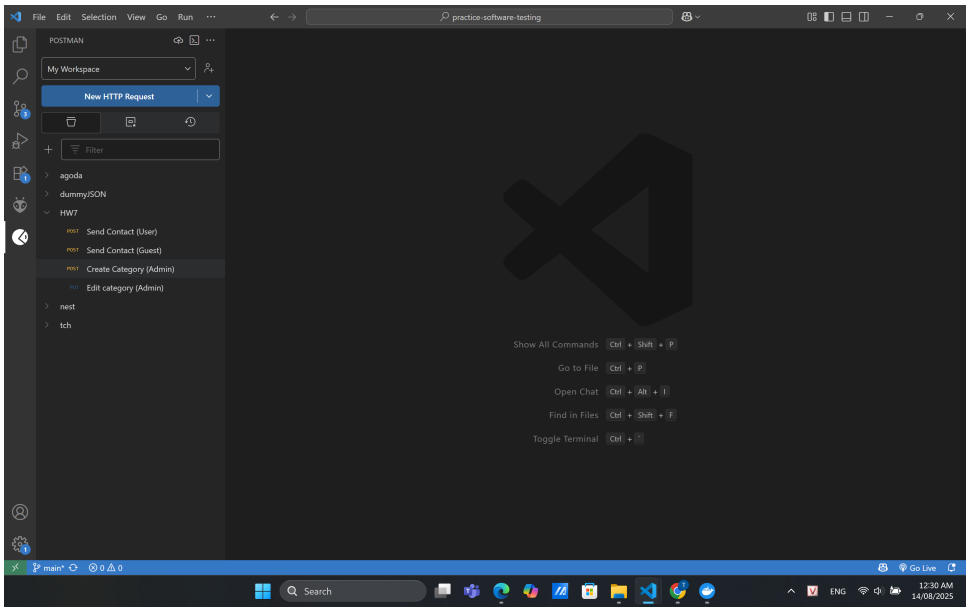

Figure 4: Create collection

2. **Log in to get tokens**

- Create a request `POST http://localhost:8091/users/login`.

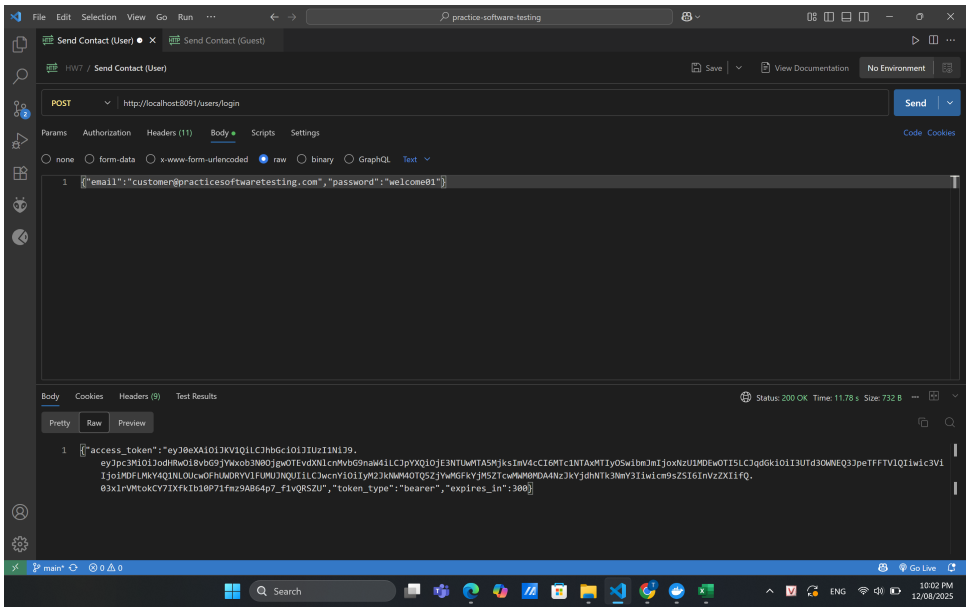

Figure 5: Login

- Click **Send**, then copy the `access_token` from the response.

3. **Set up each request**

- Open **Headers**:
  - Add `Content-Type: application/json`.

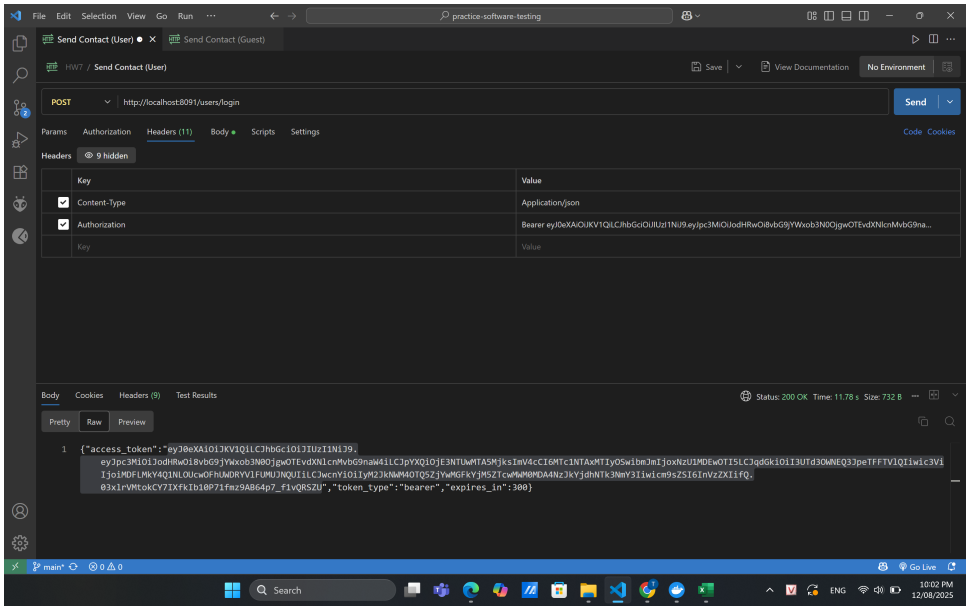

Figure 6: Save access token in header

  - If the case requires auth: `Authorization: Bearer <paste_access_token>`.
  - If it's a **guest** case, do not add `Authorization`.

- Open **Body** → **raw** → **JSON** and paste the payload from your test case.

2.4 Test Execution Summary by Feature

Feature ID	Total Cases	Passed	Failed	Pass Rate
UC01	31	20	11	64.5%
UC02	30	23	7	76.6%
UC03	30	19	11	63.3%

Overall: 91 test cases executed across 3 features, with 62 passed and 29 failed.

3 API Testing Integration into CI Workflow

3.1 Objective

The main goal is to integrate API testing into the Continuous Integration (CI) workflow so that the API test suite is executed automatically on every code change (push or pull request).

This ensures that:

- The system's APIs are continuously verified to be working as intended.
- Any breaking changes are detected as early as possible during development.
- Developers receive immediate feedback through CI logs and downloadable reports.

Initial setup:

- Forked the original repository to my own GitHub account. Forking was necessary because the original repository was not owned by me, and making changes directly could affect the original source. This approach allows working independently without impacting the upstream project.
- Cloned the forked repository to the local machine.

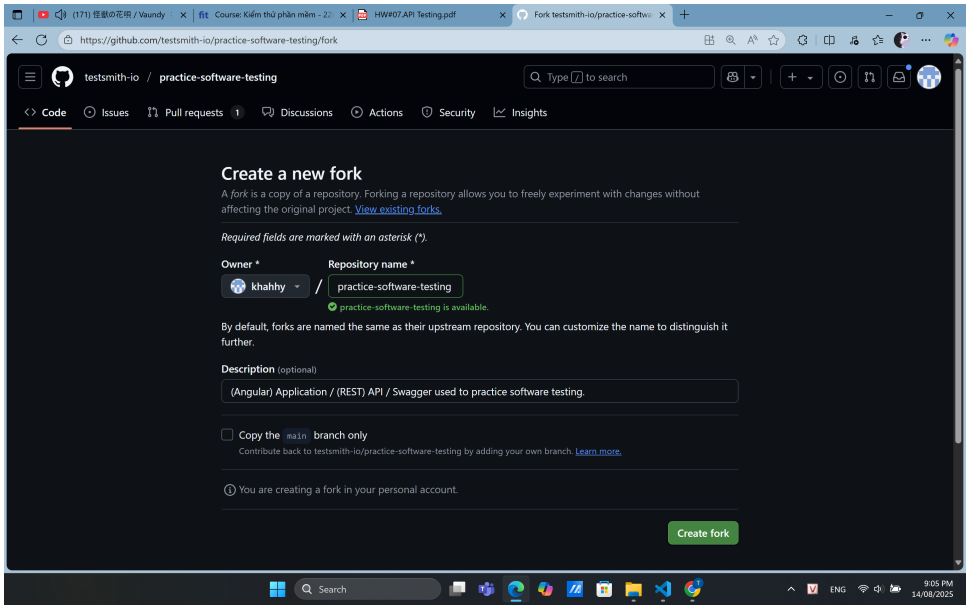

Figure 7: Forking the original repository

3.2 Implementation Steps

Step 1 – Prepare API Test Collection

We've already had the Postman collection **HW7** containing all required API endpoints for testing. Ensured that each request had the correct HTTP method, headers, body, and target URL configured.
Exported the collection in **Postman Collection v2.1** format.

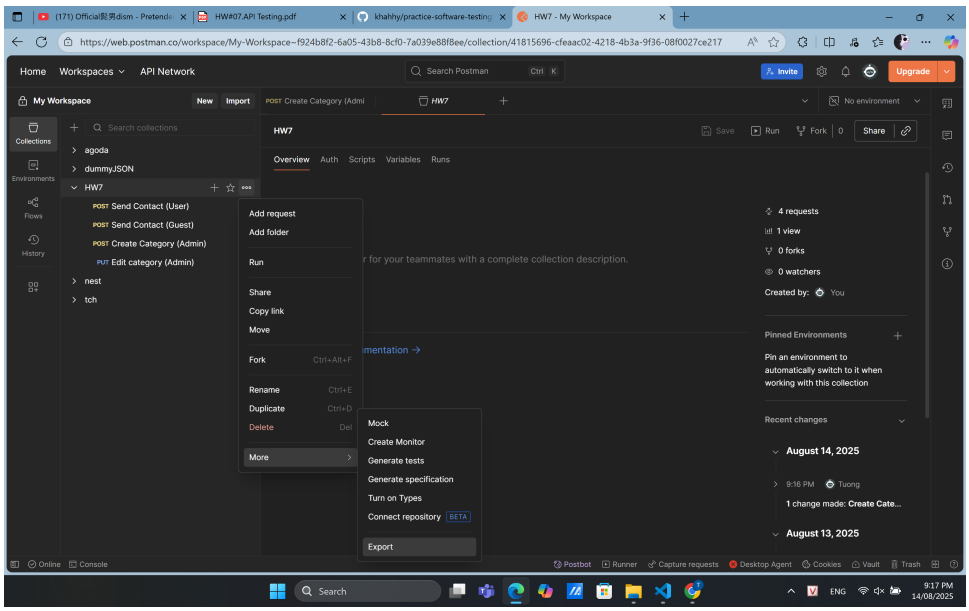

Figure 8: Exporting Postman collection

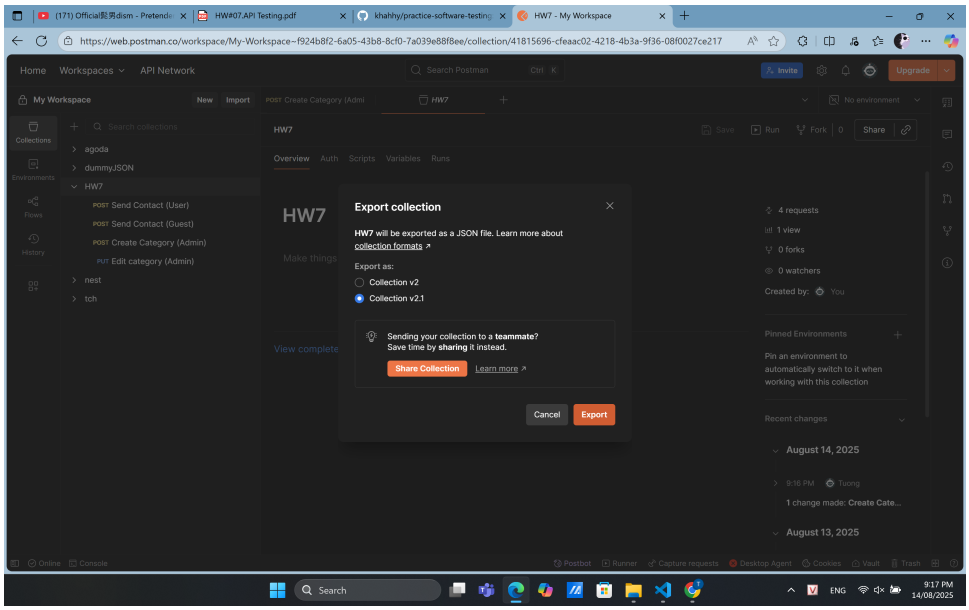

Figure 9: Choosing Collection v2.1 format

In the recently cloned source code repository, created the following folder structure to store API test resources `api-tests/collections`, placed the exported collection file into the `collections` folder with name `toolshop.postman_collection.json`

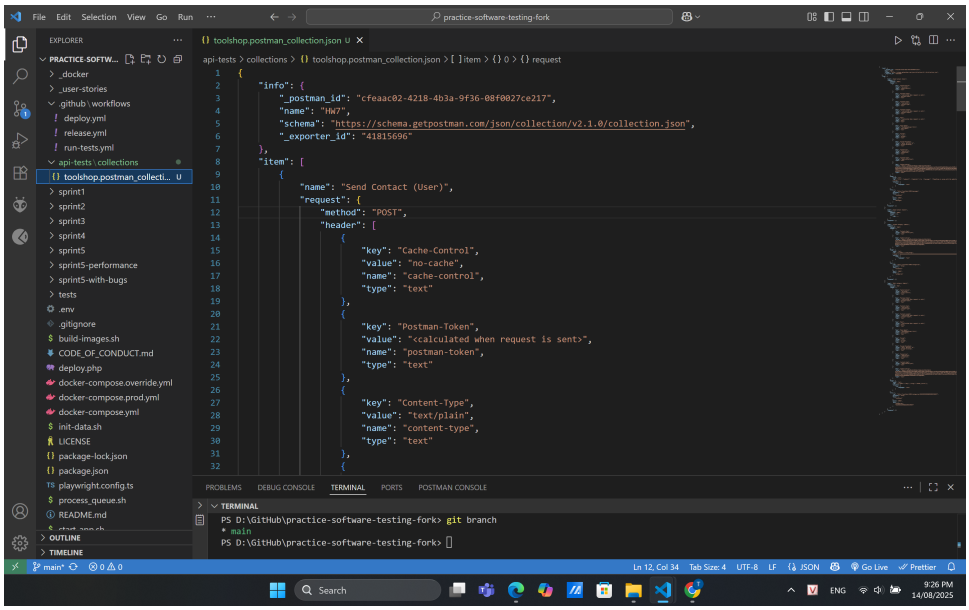

Figure 10: Postman export collection

Step 2 - Create GitHub Actions Workflow File

The cloned source repository already included three GitHub Actions workflows:

- **`run_test.yml`** – Executes UI/E2E tests using Playwright after starting the application via `docker-compose` and seeding the database.
- **`deploy.yml`** – Deploys the application to the production environment after tests pass.
- **`release.yml`** – Builds and pushes Docker images when a new release is published.

For this assignment, a new workflow file **`api-tests.yml`** was created specifically for API testing.
This workflow reused several existing steps from `run_test.yml`, such as:

- Starting the application stack with `docker-compose`.
- Seeding the database with Laravel Artisan commands.

By reusing these steps, the API test workflow could run in the same environment as the UI tests, ensuring consistency and reducing setup time.

- Created `.github/workflows/api-tests.yml`.
- Key steps in the workflow:
  1. Checkout code from the repository:

     ```yaml
     - name: Checkout
       uses: actions/checkout@v4
     ```

  2. Start the application stack using `docker-compose`:

```yaml
- name: Start containers
  run: |
    export DISABLE_LOGGING=true
    docker compose -f docker-compose.yml up -d
    --force-recreate
```

3. Seed the database using:

```
docker compose exec -T laravel-api php artisan
migrate:refresh --seed
```

4. Sanity check API via `/status` endpoint:

```yaml
- name: GET /status
  run: curl -f -X GET 'http://localhost:8091/status'
```

5. Install Newman with HTMLExtra reporter:

```yaml
- name: Install newman + reporter
  run: npm i -g newman newman-reporter-htmlextra
```

6. Run Postman collection with:

```
newman
run api-tests/collections/toolshop.postman_collection.json \
  --env-var baseUrl="http://localhost:8091" \
  --reporters cli,junit,htmlextra \
  --reporter-junit-export reports/junit.xml \
  --reporter-htmlextra-export reports/report.html
```

7. Export reports in both JUnit XML and HTML formats.

8. Upload test artifacts (`api-test-reports`) for later review:

```yaml
- name: Upload API test reports
  uses: actions/upload-artifact@v4
  with:
    name: api-test-reports
    path: reports/*
```

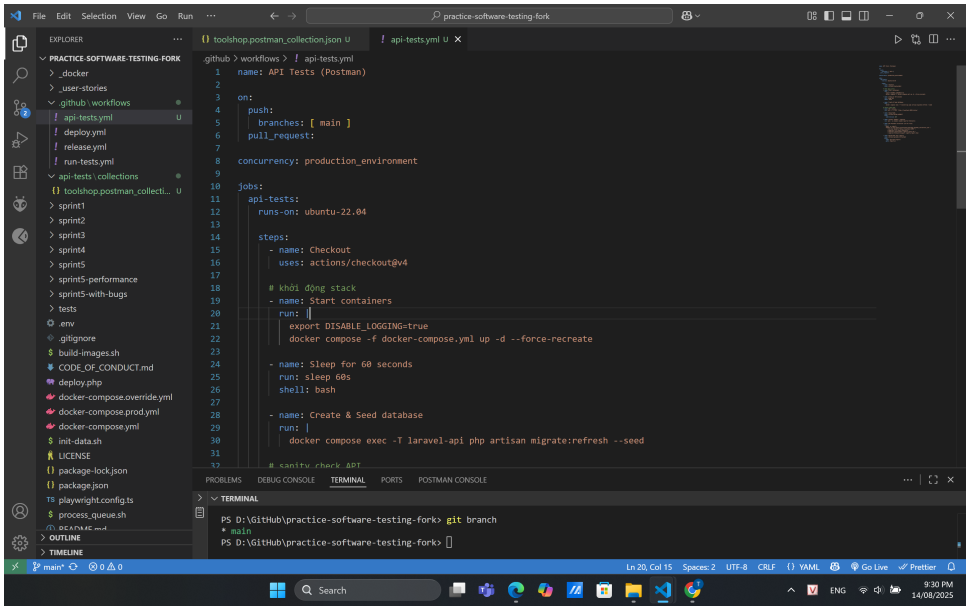

Figure 11: VS Code showing `api-tests.yml` content

Step 3 – Commit and Trigger Workflow

- Committed the Postman collection and the new workflow file `api-tests.yml` to the repository.
- Pushed the changes to the `main` branch. This automatically triggered the **API Tests (Postman)** workflow via GitHub Actions.
- The API testing workflow ran **in parallel** with other existing workflows in the repository (*Run Playwright Tests* from the original project).

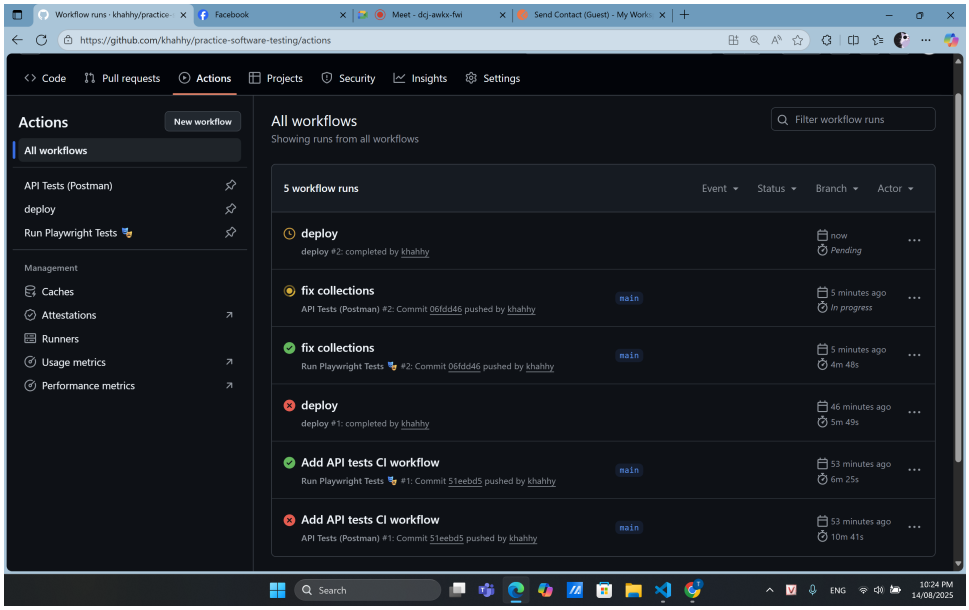

Figure 12: Actions tab showing API Tests workflow in progress

Note:

- The first commit (Add API tests CI workflow) failed due to a syntax error in the `api-tests.yml` file.

- The *Run Playwright Tests* workflow belongs to the original project and finished independently of our work.

- The **API Tests (Postman)** workflow is the one created for this assignment.

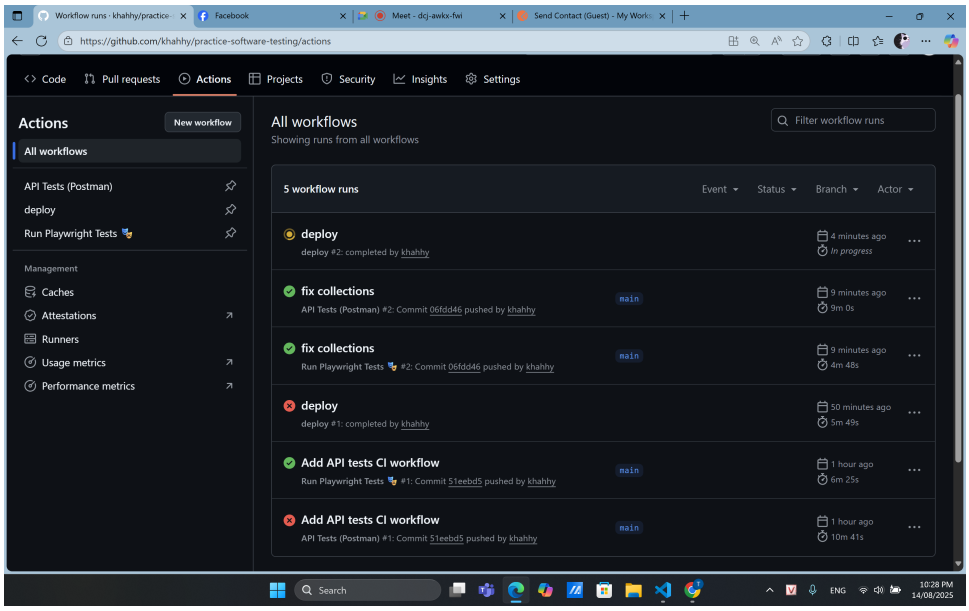

Figure 13: Actions tab showing API Tests workflow success

The **API Tests (Postman)** workflow completed successfully.

Step 4 – Results

After the workflow completed successfully, GitHub Actions produced an artifact named **`api-test-reports`**.
This artifact contains:

- **`report.html`** – A detailed Newman HTMLExtra report showing the execution results of each request, including request/response data, status codes, response times, and any failed assertions.

- **`junit.xml`** – A JUnit-compatible XML report for integration with other CI/CD tools, test dashboards, or reporting pipelines.

The generated artifact can be downloaded directly from the workflow run summary in GitHub Actions. Once downloaded and extracted, the folder structure was:

```
api-test-reports/
- junit.xml
- report.html
```

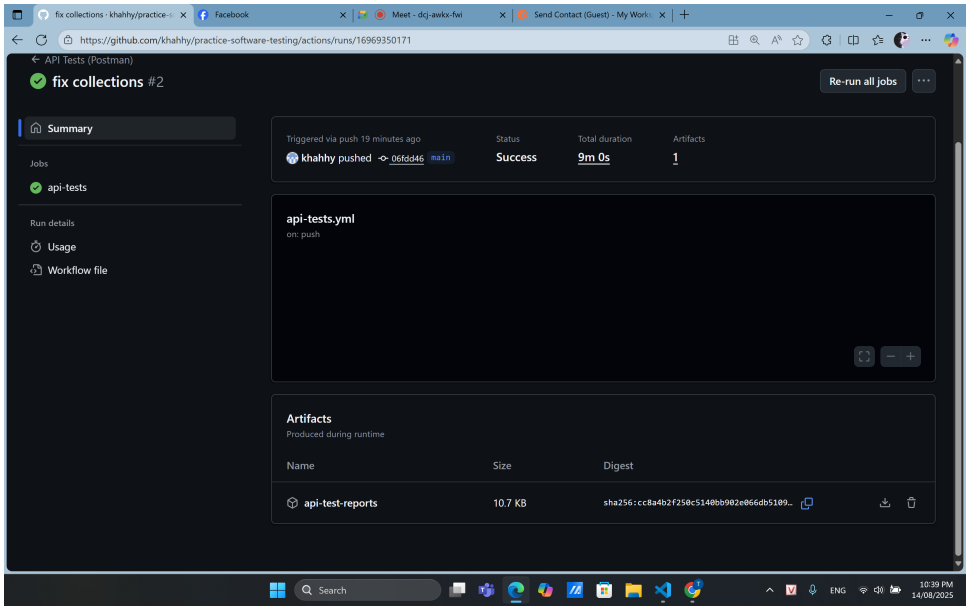

Figure 14: Artifact in GitHub Actions UI

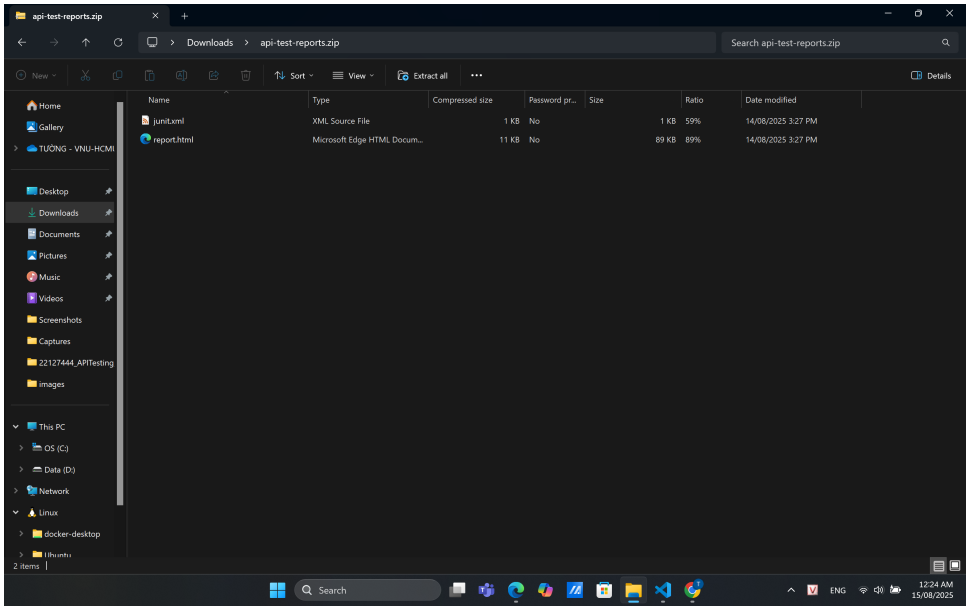

Figure 15: Extracted artifact contents

Opening `report.html` in a browser provides an interactive dashboard view, summarizing:

- Total iterations run
- Total requests executed
- Assertions passed/failed/skipped
- Performance metrics like average response time and total run duration

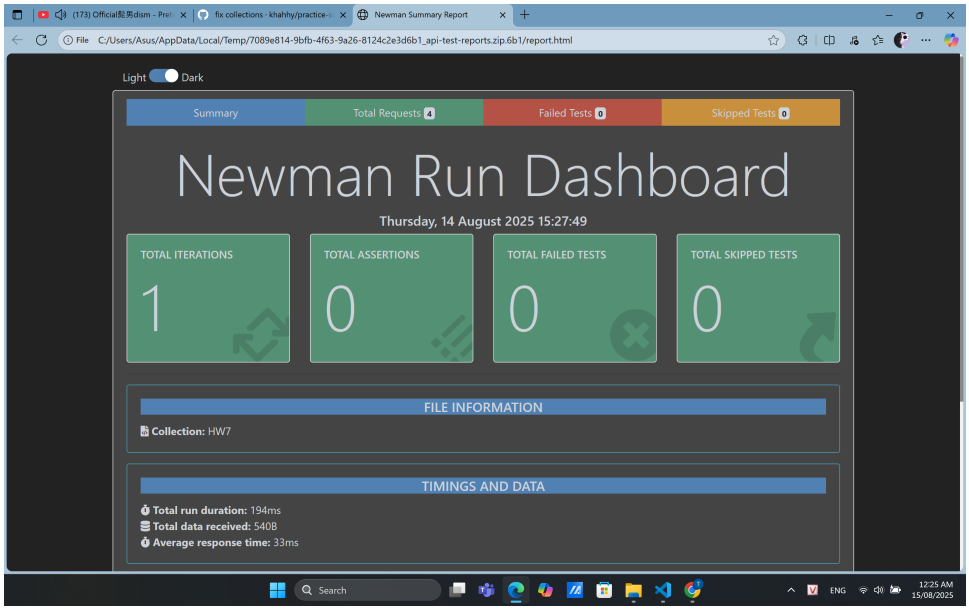

Figure 16: Newman HTMLExtra summary report

3.3 Conclusion

- Successfully integrated API testing into the CI workflow using GitHub Actions.

- On every push or pull request, the workflow:
- 1. Spins up the backend environment and seeds the database.
  2. Runs the Postman test collection using Newman.
  3. Stores test reports as artifacts for traceability.
- This setup ensures **continuous verification of API functionality** and provides **clear evidence of test execution**.

4 Self-Evaluation

Criteria	Outcomes	Percent	Self-Assessed Grade
1	**API 1**	**30%**	
1.1	Report	15	15
1.2	Test Cases (30)	5	5
1.3	Screenshots on Testing Tools	5	5
1.4	Bugs	5	5
2	**API 2**	**30%**	
2.1	Report	15	15
2.2	Test Cases (30)	5	5
2.3	Screenshots on Testing Tools	5	5
2.4	Bugs	5	5
3	**API 3**	**30%**	
3.1	Report	15	15
3.2	Test Cases (30)	5	5
3.3	Screenshots on Testing Tools	5	5
3.4	Bugs	5	5
4	**Well formatted**	**10%**	10
	Total	**100%**	**100**